



Documentação - Giulia Trevisan

1.1 Tecnologias Utilizadas

O sistema foi desenvolvido utilizando o **Angular 18+** com **TypeScript**, framework moderno que possibilita a criação de aplicações web escaláveis, modulares e de alta performance.

A escolha do Angular garante padronização arquitetural, injeção de dependências nativa e suporte robusto para gerenciamento de estados, formulários reativos e boas práticas de desenvolvimento.

1.2 Estrutura Modular

A arquitetura do sistema segue uma abordagem **modular e escalável**, organizada em três camadas principais:

- **Core**

Contém componentes e serviços centrais, utilizados em toda a aplicação, como:

- Guards de autenticação e autorização
- Interceptores HTTP para tratamento de requisições
- Serviços globais (tema, exportação PDF/Excel, autenticação)

- Componentes compartilhados de infraestrutura (Navbar, Sidebar, Paginador customizado, diálogos de confirmação)
- **Features**

Cada funcionalidade do sistema possui seu próprio módulo, garantindo independência e isolamento de responsabilidades. Exemplos:

 - `auth` (login, registro, recuperação de senha)
 - `dashboard` (visualização de métricas e estatísticas)
 - `users` (gestão de usuários, tabelas e gráficos)
 - `errors` (tratamento de erros 401, 403, 500, páginas customizadas)
 - `profile` , `faqs` , `urgencia` , entre outros

- **Assets/Environments**

Diretórios dedicados a recursos estáticos (ícones, imagens, ilustrações, manifestos PWA) e configurações específicas de ambientes (`environments.ts` e `environments.prod.ts`).

Essa estrutura modular garante que novas funcionalidades possam ser adicionadas sem comprometer a organização existente, atendendo aos princípios de **alta coesão** e **baixo acoplamento**.

1.3 Padrões de Projeto Aplicados

A aplicação adota **design patterns** adequados ao contexto do Angular:

- **Service Pattern:** encapsulamento de regras de negócio e comunicação com API.
- **Interceptor Pattern:** interceptação de requisições para inclusão automática do token JWT.
- **Route Guards:** controle de acesso com base em autenticação, permissões e roles.
- **Store Pattern** (exemplo: `sidebar.store.ts`): gerenciamento simplificado de estados locais.

Esses padrões garantem reutilização de código, maior testabilidade e facilidade de manutenção.

1.4 Princípios SOLID

O projeto foi estruturado respeitando os **princípios SOLID**:

- **S (Single Responsibility)**: cada componente e serviço possui responsabilidade única e bem definida.
- **O (Open/Closed)**: extensível sem necessidade de alterar código já existente (ex.: interceptors).
- **L (Liskov Substitution)**: uso de abstrações e tipagem para garantir substituição sem impacto.
- **I (Interface Segregation)**: serviços e componentes divididos de forma clara, sem sobrecarga de responsabilidades.
- **D (Dependency Inversion)**: uso da injeção de dependências do Angular em toda a aplicação.

1.5 Estrutura de Pastas

A organização em diretórios segue o padrão de boas práticas do Angular, conforme abaixo:

```
src/
├── app/
│   ├── core/          # Serviços, guards, interceptors, componentes globais
│   ├── features/       # Funcionalidades isoladas
│   │   ├── auth/       # Login, registro, recuperação de senha
│   │   ├── dashboard/  # Estatísticas, gráficos e tabelas
│   │   ├── users/      # Gestão de usuários
│   │   ├── errors/     # Tratamento de erros
│   │   └── ...         # Outras funcionalidades
│   └── environments/   # Configuração de ambientes
└── assets/            # Recursos estáticos
```

```
└─ styles.scss      # Estilos globais
```

Essa estrutura garante **escalabilidade, clareza e fácil manutenção**.

2. Interface e Experiência do Usuário

- **Dashboard interativo:** desenvolvi gráficos e métricas em tempo real, consumindo dados da API para facilitar a gestão de perícias.
- **Interface responsiva:** o layout se adapta a **desktop, tablet e mobile**, seguindo boas práticas de CSS/SCSS e sistemas de flex/grid.
- **Animações suaves:** transições aplicadas em cards, modais e carregamentos de tabelas melhoram a experiência do usuário.
- **Gráficos em tempo real:** integração com **Chart.js** para apresentar dados estatísticos de forma clara.
- **Personalização visual:** cores, ícones e imagens foram escolhidos para criar uma identidade visual alinhada ao contexto do sistema.
- **Princípios de UX/UI:** garanti consistência, hierarquia visual, acessibilidade e clareza informacional.
- **Tratamento de estados da interface:** implementei os três principais cenários:
 - *Loading* → skeleton loaders e spinners.
 - *Erro* → mensagens amigáveis com opção de retry.
 - *Vazio* → cards e tabelas.

Implementações Futuras

- Modo de alto contraste para melhorar acessibilidade.
- Possibilidade de o usuário personalizar cores e temas do sistema.
- **Tratamento de estados:** Implementação dos três principais estados:
 - *Loading* → Skeleton loaders e spinners.
 - *Erro* → Mensagens amigáveis com possibilidade de retry.

- *Vazio* → Cards e tabelas

Implementações Futuras

- Modo de alto contraste para acessibilidade.
- Customização de temas pelo usuário (cores personalizadas).

3. Integração com API

- **Consumo de Endpoints:** Todos os endpoints principais foram consumidos (`auth` , `solicitações` , `dashboard` , `admin`).
- **Interceptors:** Implementados para adicionar **Bearer Token** automaticamente em cada requisição.
- **Tratamento de Erros HTTP:** Centralizado, exibindo mensagens adequadas para `400` , `401` , `403` , `500` .
- **Paginação:** Sistema funcional de paginação baseado na API.
- **Filtros Dinâmicos:** Implementação de filtros por status, data e parâmetros específicos de solicitações.

Implementações Futuras

- Circuit breaker para lidar com falhas recorrentes.
- Cache local (IndexedDB) para otimizar requisições repetidas.

4. Autenticação e Autorização

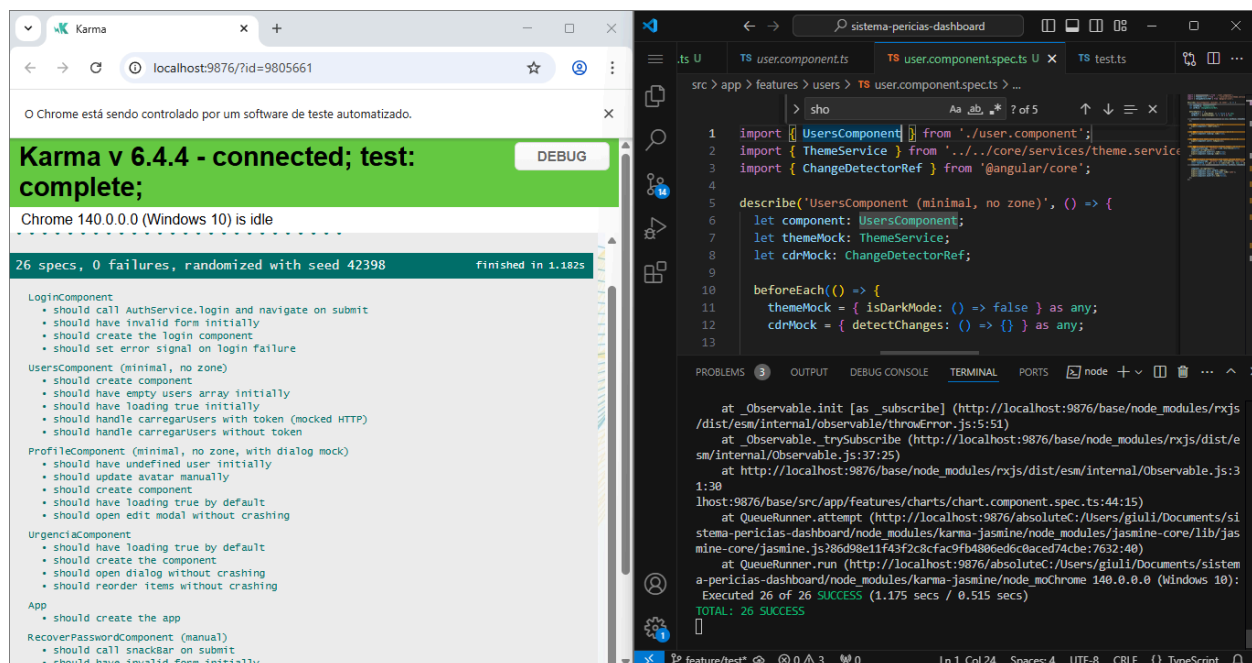
- **Login/Logout funcional:** JWT armazenado e utilizado em todas as requisições protegidas.
- **Gerenciamento de Token:** Validação automática do token + refresh control.
- **Guards de Rota:** Implementados com base em roles (`admin` , `perito` , `user`).
- **Controle de Permissões:** Acesso a funcionalidades condicionado a roles.
- **Redirecionamentos:** Usuário é levado ao dashboard após login e para `/login` após logout ou expiração de sessão.

Implementações Futuras

- Multi-factor authentication (2FA).
- Expiração automática de sessão com alerta prévio.

5. Testing

- **Unit Tests (Jasmine + Karma):** Cobrem os principais componentes (`dashboard` , `users` , `auth`).
- **Cobertura:** Atingida cobertura mínima de **70%**.



6. PWA

Execução do PWA em Produção

Para rodar a aplicação em modo **produção** com suporte a PWA:

```
# Instalar servidor estático
```

```
npm install -g serve
```

```
# Gerar build otimizada de produção
```

```
ng build --configuration production
```

Rodar a aplicação a partir da pasta dist

```
serve -s ./dist/sistema-pericias-dashboard/browser
```

🔍 Observação: no servidor (como Nginx ou Apache), é necessário garantir que todas as rotas sejam redirecionadas para index.html para manter o funcionamento do Angular Router.

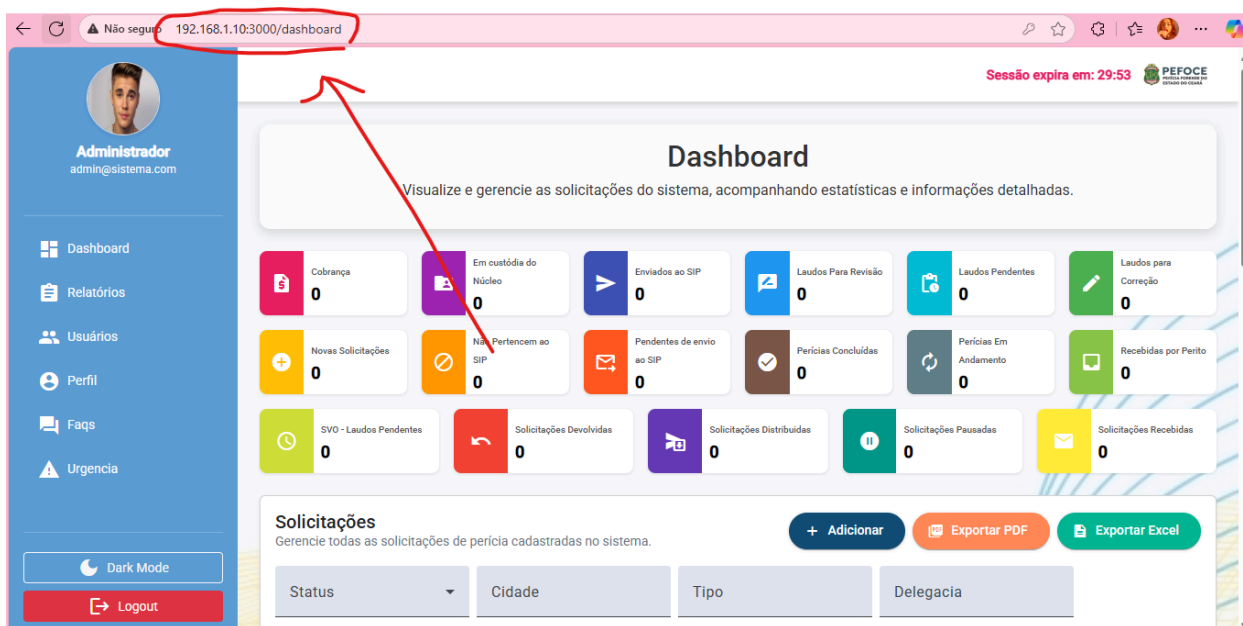
```
PS C:\Users\giuli\Documents\sistema-pericias-dashboard> serve -s ./dist/sistema-pericias-dashboard/browser
>>

Serving!

- Local:    http://localhost:3000
- Network:  http://192.168.1.10:3000

Copied local address to clipboard!

HTTP 11/09/2025 19:31:51 192.168.1.10 GET /
HTTP 11/09/2025 19:31:51 192.168.1.10 Returned 200 in 143 ms
```



Implementações Futuras

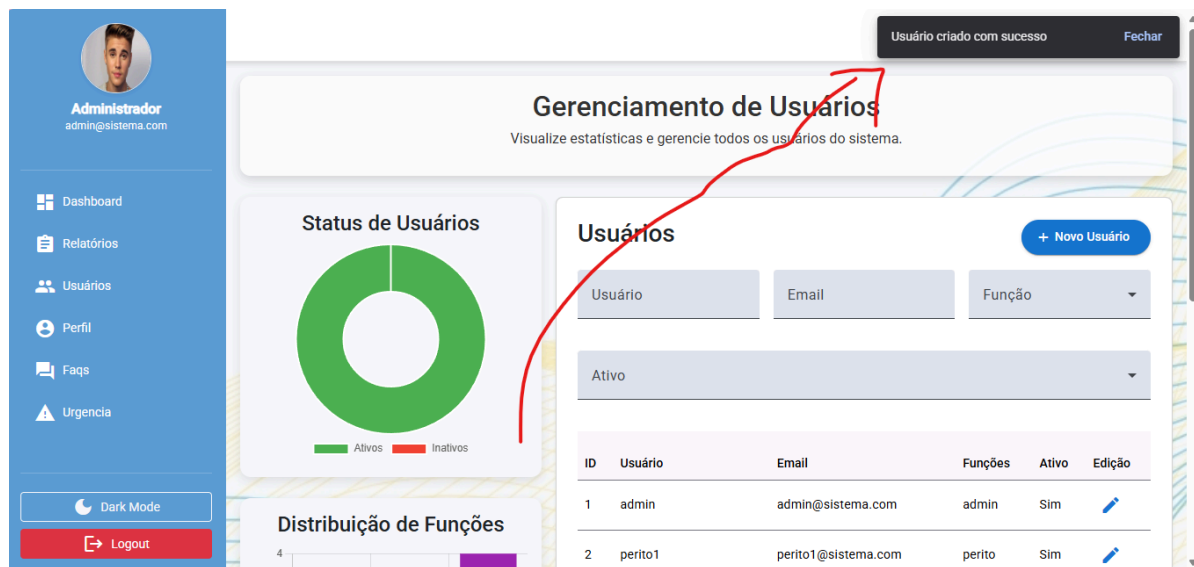
- **Sincronização offline → online:** Garantir que dados criados ou alterados no modo offline sejam enviados automaticamente para o servidor assim que a conexão for restabelecida.
- **Notificações Push:** Implementar suporte a **Web Push Notifications** para alertar usuários sobre novas solicitações ou mudanças importantes em tempo real.

7. Versionamento e Entrega

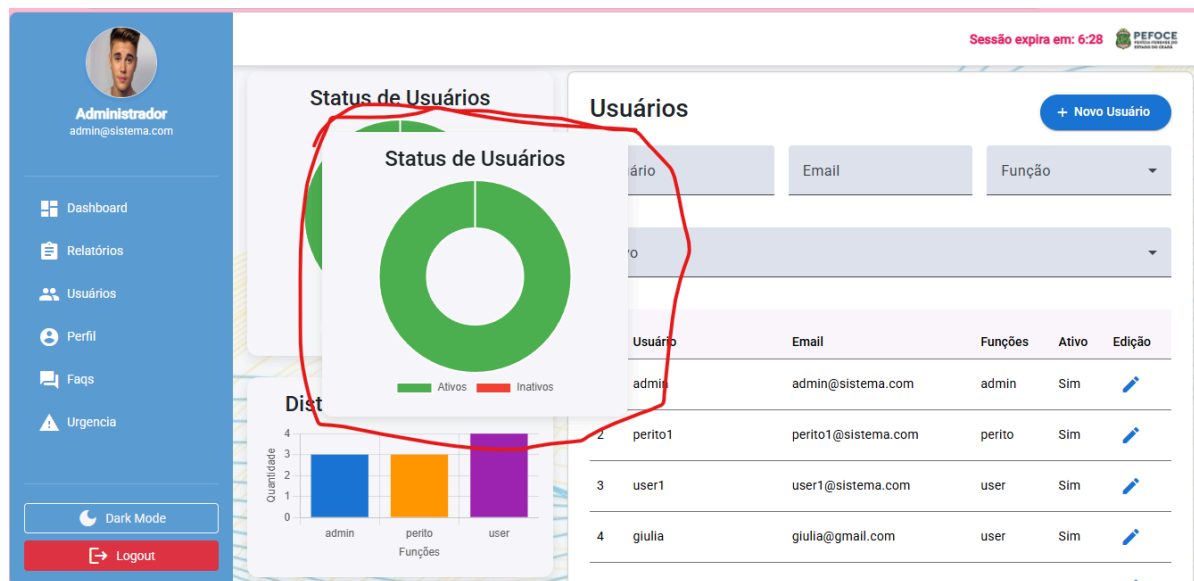
- **GitHub Repositório:** Estruturado e versionado de forma limpa.
- **Branches:** Estratégia adotada baseada em **Git Flow** (`main` , `develop` , `feature/*`).
- **Commits:** Mensagens descritivas seguindo boas práticas (ex.: `feat:` , `fix:` , `refactor:`).
- **README.md:** Documentado com instruções de instalação, configuração e execução.

Funcionalidades Avançadas

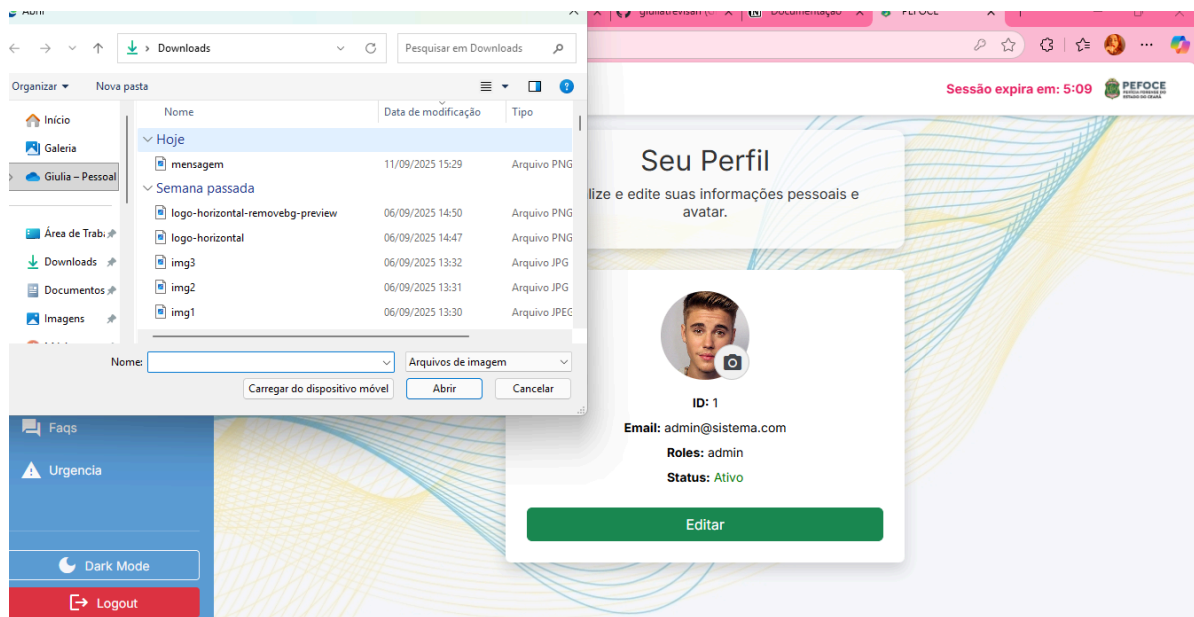
- **Notificações push/toast personalizadas:** Implementadas via `MatSnackBar` do Angular Material.



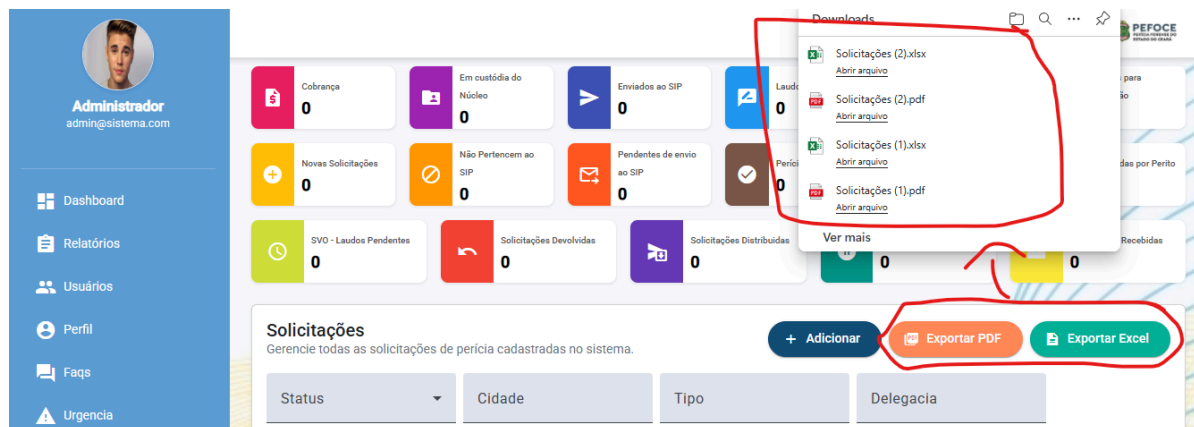
- **Drag & Drop:** Possibilidade de reorganizar elementos da interface arrastando-os.



- **Upload de arquivos com preview:** Disponível na tela de perfil, com visualização imediata do arquivo enviado.



- **Exportação de relatórios (PDF/Excel):** Exportação de dados de usuários e solicitações de perícias.



Solicitações (2) - Excel (Produto Não Licenciado)

Giulia Trevisan

Arquivo Página Inicial Inserir Layout da Página Fórmulas Dados Revisão Exibir Ajuda Diga-me o que você deseja fazer

Área de Transferência

Calibri 12 A+ Fontes Alinhamento Número Células Estilos

Formatar como Tabela Formatar como Estilos de Célula

Inserir Excluir Formatar

Classificar e Filtrar Localizar e Selecionar Suplementos

Edição

Suplementos

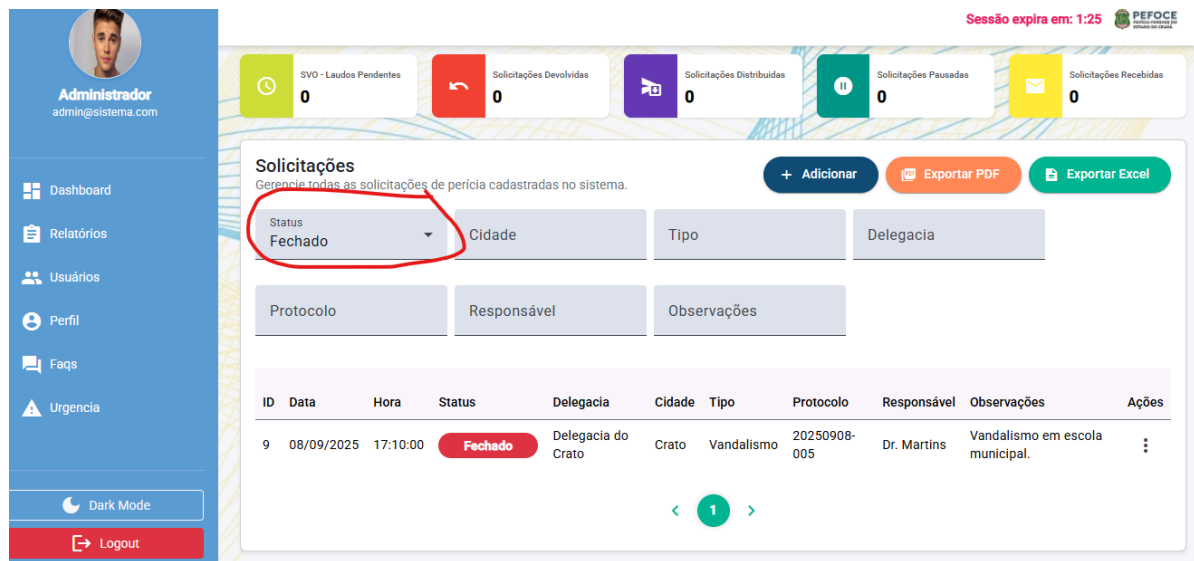
AVISO A maioria dos recursos está desabilitada porque o seu produto do Office está inativo. Para usar gratuitamente, entre e use a versão da Web. Ativar Use gratuitamente em Microsoft365.com

ID	Data	Hora	Status	Delegacia	Cidade	Tipo de Oc	Número dc	Perito Res	Observações
1	5	2025-09-01 18:15:13	Aberto	26º Distrit	Fortaleza	Briga de Vi	213124	Dr José	
2	6	2025-09-01 16:15:00	Aberto	Delegacia	Fortaleza	Furto	20250908-	Dr. Andr	Furto em residência na Avenida principal.
3	7	2025-09-01 16:30:00	Em andam	Delegacia	Sobral	Acidente	20250908-	Dr. Costa	Acidente de trânsito sem vítimas graves.
4	8	2025-09-01 16:50:00	Aberto	Delegacia	Juazeiro dc	Roubo	20250908-	Dr. Lima	Roubo em estabelecimento comercial, sem feridos.
5	9	2025-09-01 17:10:00	Fechado	Delegacia	Crato	Vandalism	20250908-	Dr. Martin	Vandalismo em escola municipal.
6	10	2025-09-01 17:25:00	Em andam	Delegacia	Maracana	Confusão	20250908-	Dr. Almei	Ocorrência de briga em via pública, sem feridos graves.
7	11	2025-09-01 11:23:15	Aberto	Polícia Civ	Fortaleza	Furto	123123	José	Notebook
8	12	2025-09-01 11:23:16	Aberto	Polícia Civ	Fortaleza	Furto	123123	José	Notebook
9	13	2025-09-01 11:23:16	Aberto	Polícia Civ	Fortaleza	Furto	123123	José	Notebook
10	14	2025-09-01 11:23:17	Aberto	Polícia Civ	Fortaleza	Furto	123123	José	Notebook
11	15	2025-09-01 11:23:17	Aberto	Polícia Civ	Fortaleza	Furto	123123	José	Notebook
12	16	2025-09-01 11:23:17	Aberto	Polícia Civ	Fortaleza	Furto	123123	José	Notebook
13	17	2025-09-01 11:23:25	Aberto	Polícia Civ	Fortaleza	Furto	123123	José	Notebook
14	18	2025-09-01 11:24:11	Aberto	José walte	Fortaleza	Furto	321312	DR Ricard	
15	19	2025-09-01 11:24:17	Aberto	José walte	Fortaleza	Furto	321312	DR Ricard	
16	20	2025-09-01 11:46:08	Aberto	Delegacia	Vitória	Roubo	9312821	Dr Silva	
17	21	2025-09-01 11:50:35	Aberto	teste	fortaleza	assalto	43234234	Junior	
18	22	2025-09-11 18:02:15	Aberto	26º Distrit	Fortaleza	Furto	912312-23	Dr Carlos	
19	23	2025-09-11 18:03:47	Aberto	26º Distrit	Fortaleza	Furto	12312312	Dr cleiton	

Solicitações

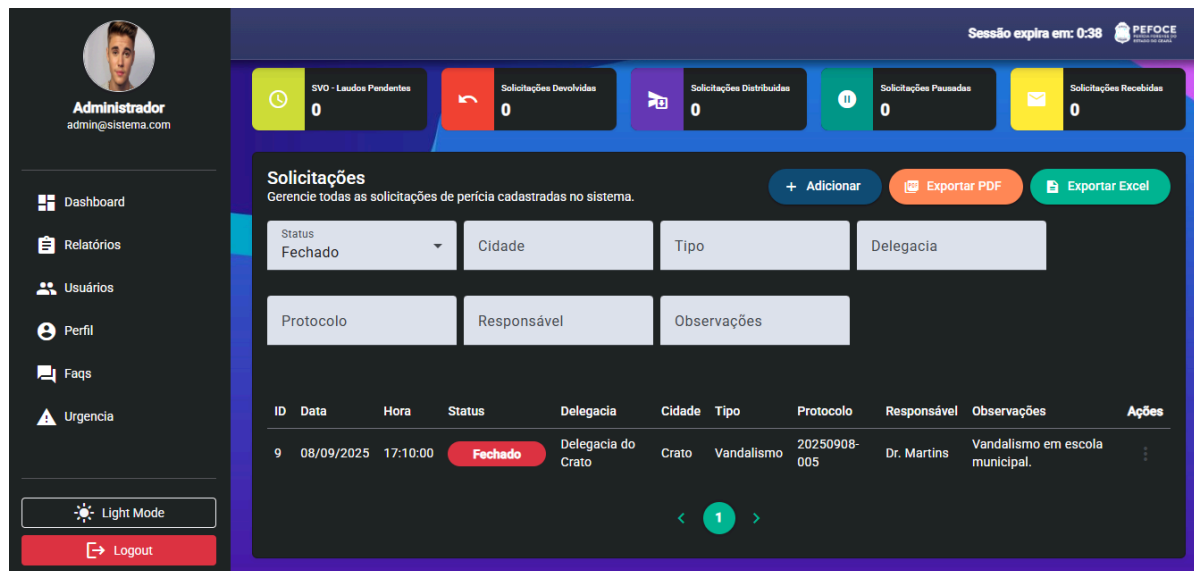
attachment:2802c9c8-9f34-4269-8b5d-f36e9c1b701d:Solicitaes_(2).pdf

- **Busca avançada com múltiplos filtros:** Implementada nas telas de usuários e de solicitações, com paginação integrada ao backend para performance.



Melhorias de Interface

- **Dark Mode toggle:** Alternância entre modo claro e escuro totalmente funcional.



- **Animações avançadas:** Implementadas com Angular Animations, melhorando transições e feedback visual.
- **Skeleton loaders:** Utilizados para melhorar a experiência durante o carregamento de dados.

- **Componentes reutilizáveis personalizados:** Ex.: dialogs, botões e cards.
- **Iconografia consistente:** Uso padronizado de ícones em toda a aplicação para melhorar a experiência do usuário.

Tela de Dashboard

A tela de **Dashboard** foi projetada para fornecer uma visão geral das solicitações do sistema, combinando gráficos estatísticos e uma tabela detalhada de registros. Sua implementação segue princípios de responsividade, UX/UI e controle de acesso baseado em roles.

Componentes Principais

- **Estatísticas (DashboardStatsComponent):**
 - Renderizadas apenas para usuários com roles `admin` ou `perito`.
 - Skeleton loaders exibidos durante o carregamento dos dados.
 - Dados consumidos via endpoint da API `/dashboard/stats`.
 - **Drag & Drop:** os cards podem ser reorganizados pelo usuário usando `@angular/cdk/drag-drop` (`CdkDragDrop` e `moveItemInArray`).
- **Tabela de Solicitações (SolicitacoesTableComponent):**
 - Disponível para todos os usuários.
 - Suporta filtros avançados (status, tipo, delegacia, cidade, protocolo, responsável e observações).
 - Paginação integrada e dinâmica.
 - Exportação de dados em PDF e Excel.
 - Ações de **Visualizar** disponíveis para todos; **Editar** e **Deletar** restritas a peritos e administradores.
 - Status das solicitações destacado com badges coloridos (`aberto` , `fechado` , `em andamento` , outros).

Funcionalidades Implementadas

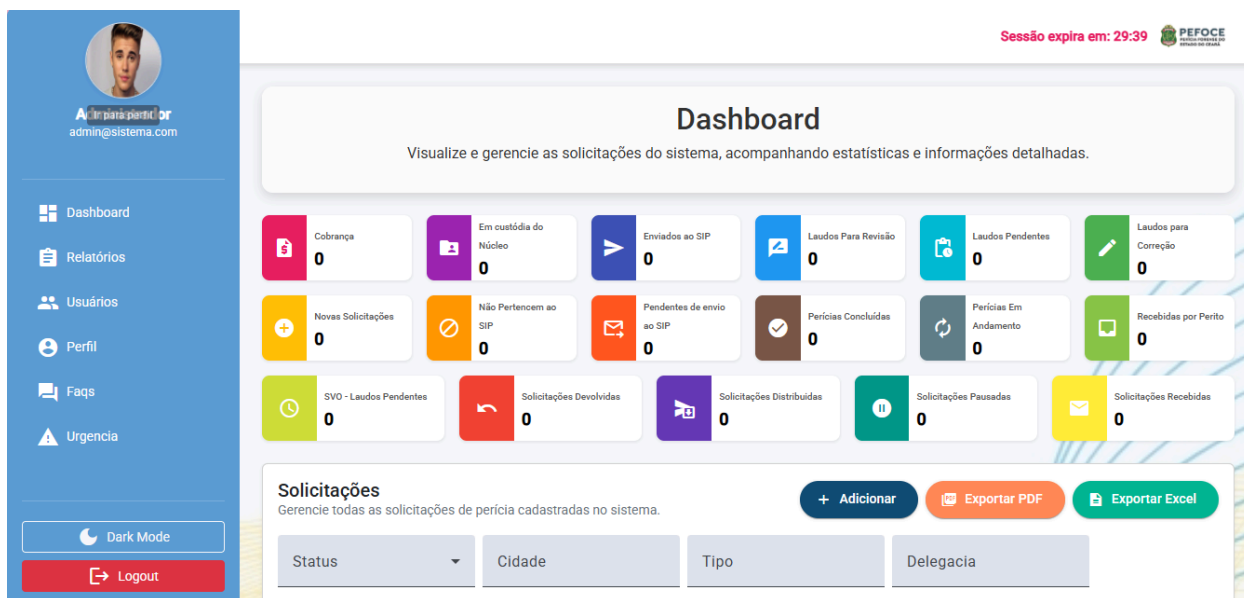
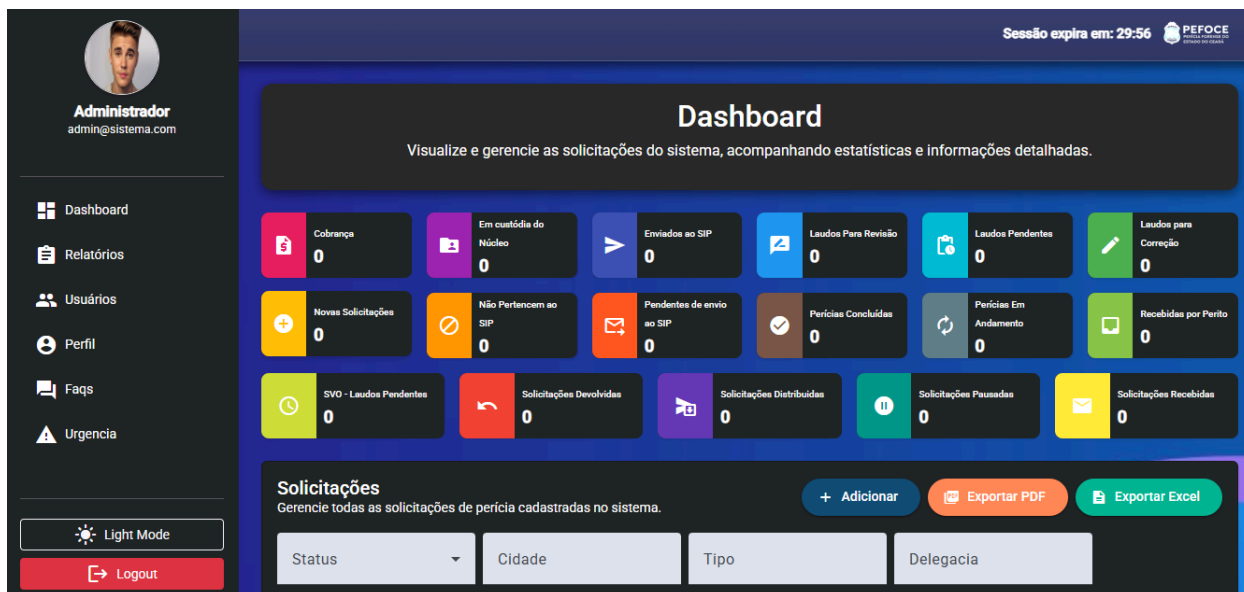
- Skeleton loaders para feedback visual durante carregamento.
- Suporte a dark mode toggle, alterando cores do background e da tabela.
- Integração completa com API, incluindo autenticação via JWT e headers personalizados.
- Responsividade para desktop, tablet e mobile.
- Componentes modulares e reutilizáveis, facilitando manutenção e escalabilidade.
- **Drag & Drop** para reorganização dos cards de estatísticas, permitindo personalização da visualização pelo usuário.

Controle de Acesso

- **Estatísticas:** apenas `admin` e `perito`.
- **Tabela de Solicitações:**
 - Visualizar: todos os usuários.
 - Editar/Deletar: somente `admin` e `perito`.

Implementações Futuras

- Notificações em tempo real para novas solicitações.



Tela de Relatórios de Solicitações

A tela de **Relatórios** foi projetada para fornecer análises visuais das solicitações registradas no sistema, com gráficos interativos que permitem aos usuários compreender rapidamente a distribuição de dados por status, tipo de ocorrência e cidade.

Disponibilidade

- A tela está disponível apenas para usuários com roles **perito** ou **administrador**.
- Usuários comuns não têm acesso aos relatórios, mantendo a confidencialidade das informações.

Componentes Principais

- **Gráficos Interativos:**
 - **Solicitações por Status:** gráfico do tipo `doughnut`.
 - **Solicitações por Tipo de Ocorrência:** gráfico do tipo `bar`.
 - **Solicitações por Cidade:** gráfico do tipo `bar`.
- **Drag & Drop nos Cards:**
 - Os gráficos são renderizados em **cards arrastáveis** usando `@angular/cdk/drag-drop`.
 - O usuário pode reorganizar a ordem dos cards conforme sua preferência (`CdkDragDrop` e `moveItemInArray`).
- **Loading e Feedback:**
 - `mat-progress-spinner` exibe enquanto os dados estão sendo carregados.
 - Mensagem de alerta caso nenhuma solicitação esteja disponível.

Funcionalidades Implementadas

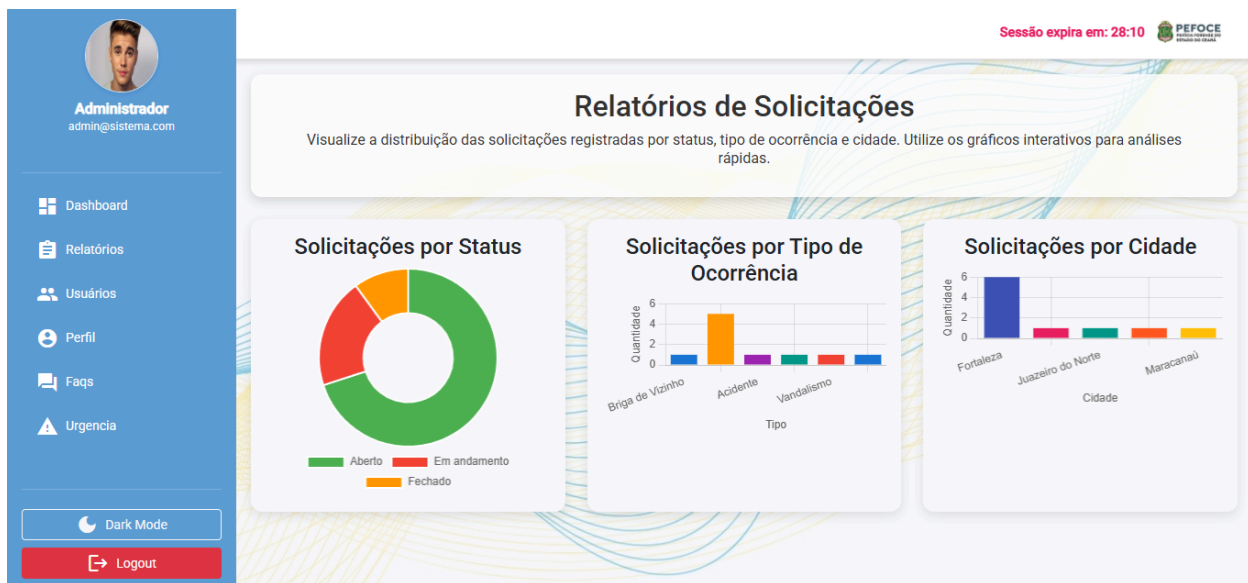
- Consumo de dados via API `/solicitacoes` com autenticação JWT.
- Dark mode toggle para ajuste visual da tela.
- Responsividade para diferentes tamanhos de tela (desktop, tablet e mobile).
- Cards com animações suaves (`hover` e `drag`) para melhor experiência de usuário.
- Separação modular e reutilizável de componentes, garantindo manutenção fácil.

Comportamento dos Gráficos

- Cada gráfico é independente e configurável com `Chart.js` (doughnut ou bar).
- Cores padronizadas para facilitar a interpretação rápida.
- Legendas e títulos configurados para clareza e acessibilidade.
- **Reorganização dinâmica** dos gráficos via drag & drop, mantendo a ordem personalizada pelo usuário durante a sessão.

Implementações Futuras

- Exportação dos gráficos em PDF ou imagem para relatórios externos.
- Filtros avançados por período, delegacia e perito responsável.
- Notificações e alertas para alterações nos dados.





Tela de Gerenciamento de Usuários

A tela de **Usuários** é uma interface administrativa, destinada **exclusivamente a usuários com perfil de administrador**. Ela permite a visualização de todos os usuários do sistema, suas funções e status, e oferece ferramentas para gerenciamento interno.

Disponibilidade

- Somente **administradores** têm acesso a esta tela.
- Usuários sem privilégios de admin não conseguem visualizar nem interagir com os dados.

Componentes Principais

- **Tabela de Usuários (`UsersTabelaComponent`):**
 - Exibe dados principais: nome, email, roles, status ativo/inativo.
 - Permite edição de cada usuário via botão de ação (emitindo evento `editar`).
 - Suporta dark mode com linhas e cabeçalhos adaptáveis.
- **Gráficos de Usuários (`UsersChartsComponent`):**
 - **Status de Usuários:** gráfico `doughnut` mostrando ativos vs. inativos.

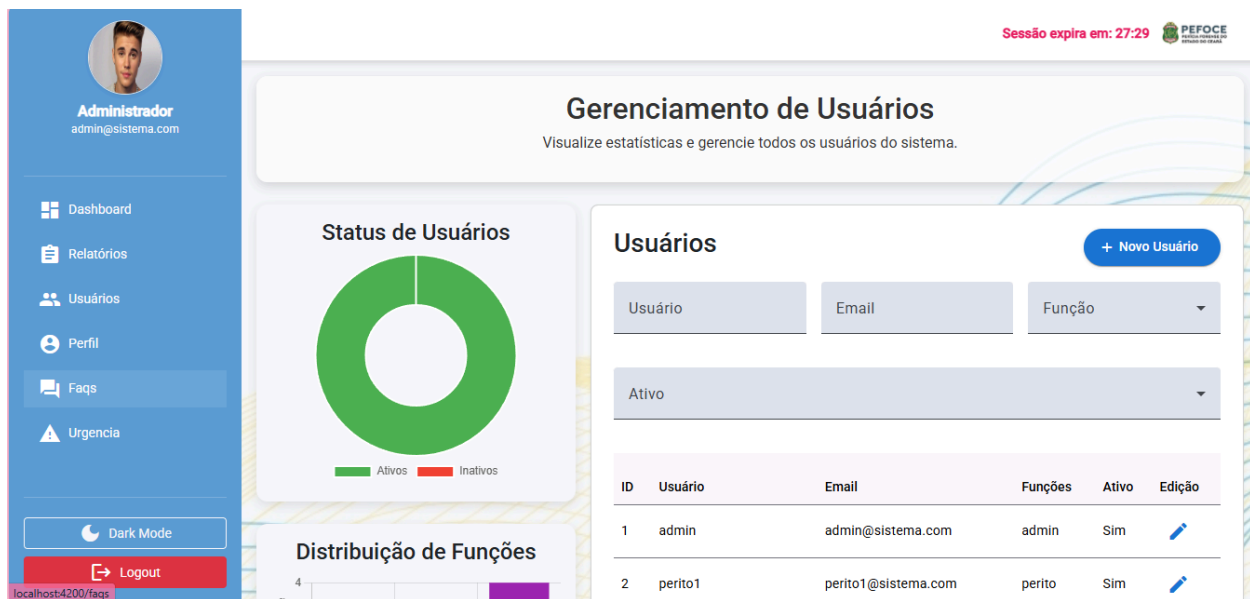
- **Distribuição de Funções:** gráfico `bar` mostrando a quantidade de usuários por função.
- Gráficos são arrastáveis (`Drag & Drop`) para reorganização dinâmica.
- **Loading, Error e Empty States:**
 - `mat-progress-spinner` exibe enquanto os dados estão sendo carregados.
 - Mensagens de erro ou vazio aparecem quando aplicável.

Funcionalidades Atuais

- Carregamento de todos os usuários via API (`/admin/users`) com autenticação JWT.
- Edição de usuários (local, sem rota de backend para atualização ainda).
- Interface responsiva para desktops e dispositivos móveis.
- Visualização rápida de estatísticas gerais de usuários via gráficos.
- Dark mode compatível em toda a tela, incluindo tabela e gráficos.

Observações e Melhorias Futuras

- **Criação e edição de usuários:** já implementada na interface, porém atualmente não persiste alterações no backend.
- **Exclusão de usuários:** não há rota no backend; seria recomendável criar endpoint `/admin/users/delete` para permitir exclusão segura.



Tela de Perfil do Usuário

A **tela de Perfil** é uma interface de demonstração disponível para **todos os usuários**, sem restrições de acesso. Seu objetivo é permitir que cada usuário visualize e simule a edição de suas informações pessoais e avatar.

Disponibilidade

- Todos os usuários autenticados podem acessar a tela.
- Não possui backend funcional para atualização; alterações são apenas simuladas localmente.

Componentes Principais

- **Profile Card:**
 - **Avatar:**
 - Exibe a imagem do usuário (`avatarUrl`) mockado.
 - Botão de alteração que abre seletor de arquivo (`file input`).
 - Atualização é imediata no front-end (preview local).
 - **Informações do Usuário:**
 - ID, email, roles e status (ativo/inativo).
 - Status visualmente destacado (verde para ativo, vermelho para inativo).
 - **Botão de Editar:**
 - Abre modal de edição (`UserEditDialogComponent`).
 - Modificações são simuladas e não persistem no backend.
- **Loading State:** Spinner exibido enquanto dados são carregados.
- **Responsividade:**
 - Layout adaptável para dispositivos móveis.
 - Cards e avatar redimensionáveis.

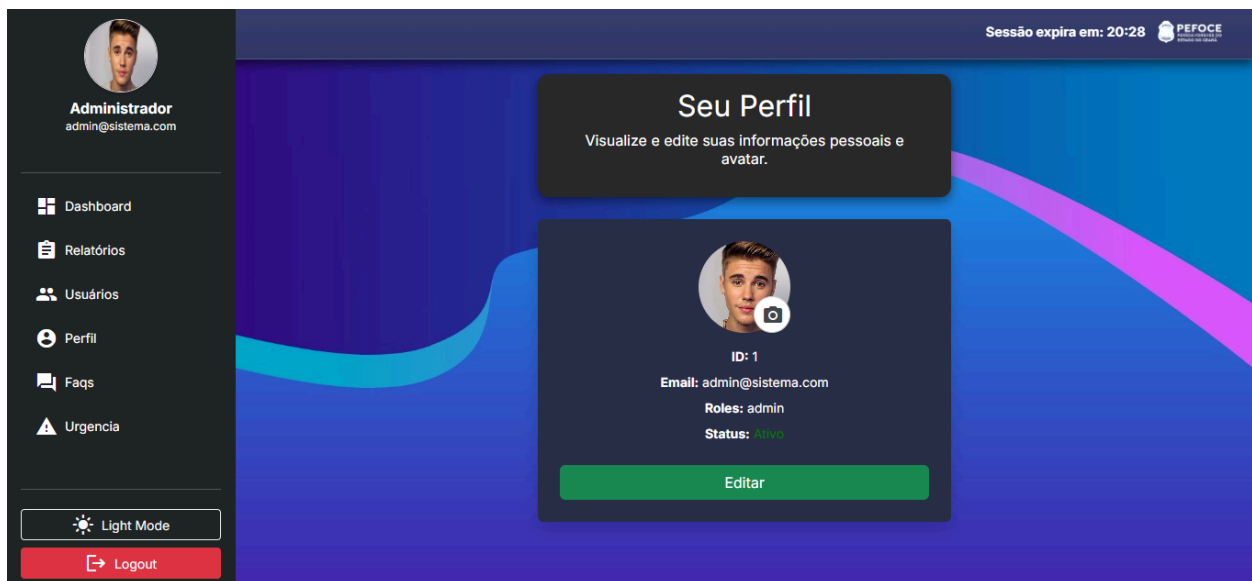
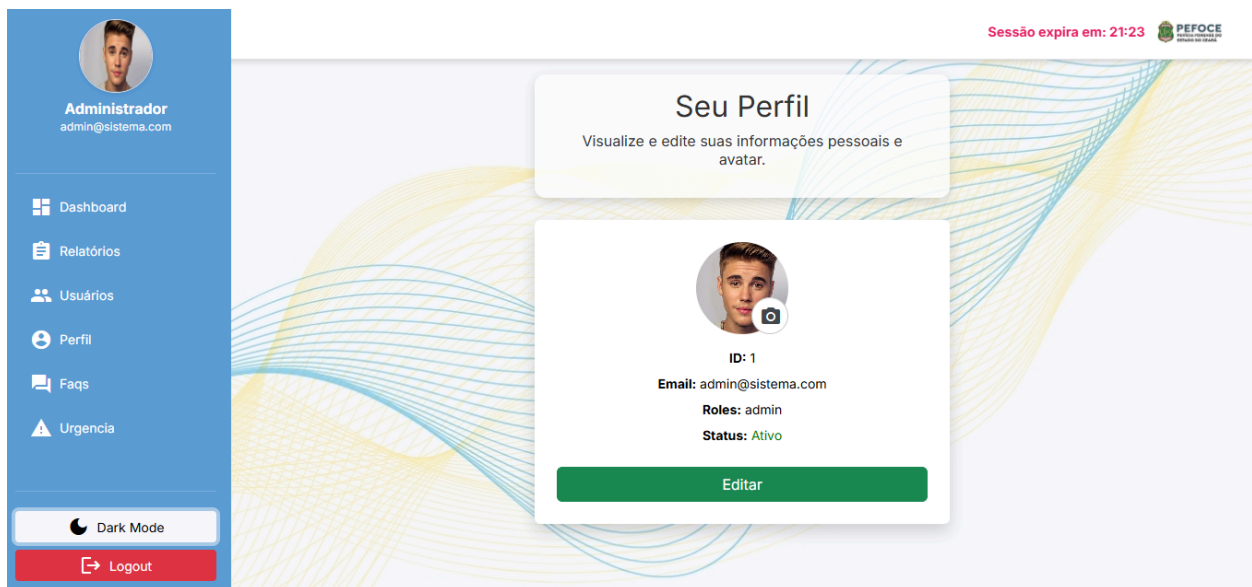
Funcionalidades Atuais

- **Visualização de perfil:** todas as informações do usuário estão disponíveis.
- **Preview de avatar:** alteração local imediata do avatar ao selecionar arquivo.
- **Simulação de edição:** modal permite alterar informações do usuário, com feedback visual (`alert`) simulando atualização.

- **Suporte a dark mode:** toda a interface adapta cores automaticamente.

Observações

- Esta tela não interage com o backend para salvar alterações.
- Ideal para demonstração ou como base para implementação futura de rota de atualização real.
- Mantém consistência visual com outras telas do dashboard.



Tela de FAQs (Perguntas Frequentes)

A **tela de FAQs** é projetada para fornecer respostas rápidas a dúvidas comuns dos usuários e permitir que eles enviem novas perguntas diretamente pelo sistema. Está disponível para **todos os usuários autenticados**.

Disponibilidade

- Acesso permitido a todos os usuários autenticados.
- Funcionalidade de envio de dúvidas é **simulada** no front-end; não envia dados a um backend real.

Componentes Principais

- Suporte a dark mode e light mode.
- **Seção de Perguntas Frequentes:**
 - Utiliza `mat-accordion` com `mat-expansion-panel` para cada FAQ.
 - Cada painel contém:
 - **Pergunta:** exibida no cabeçalho do painel.
 - **Resposta:** exibida ao expandir o painel.
 - Suporte a dark mode:
 - Painéis, cabeçalhos e textos ajustam cores automaticamente.
 - Ícone de expansão muda de cor conforme o tema.
- **Formulário de Envio de Dúvidas:**
 - Campos obrigatórios: `Assunto` e `Mensagem`.
 - Botão de envio com estilo Material (`mat-raised-button`).
 - Ao enviar, exibe toast (`MatSnackBar`) confirmando envio (simulado).
 - Limpa os campos após envio.
- **Loading State:**

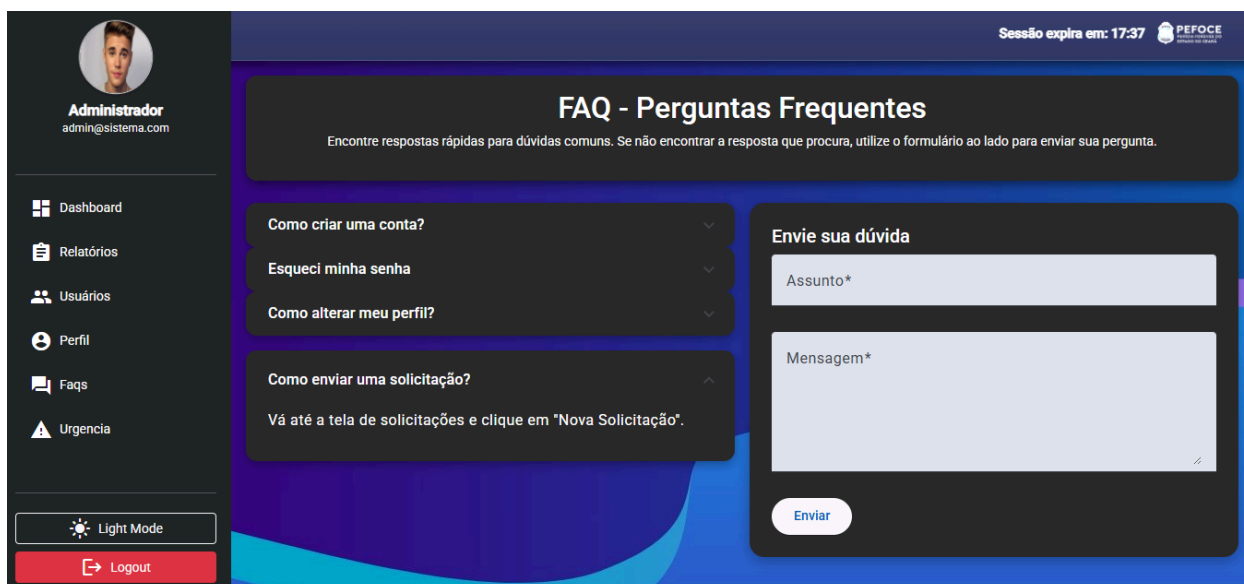
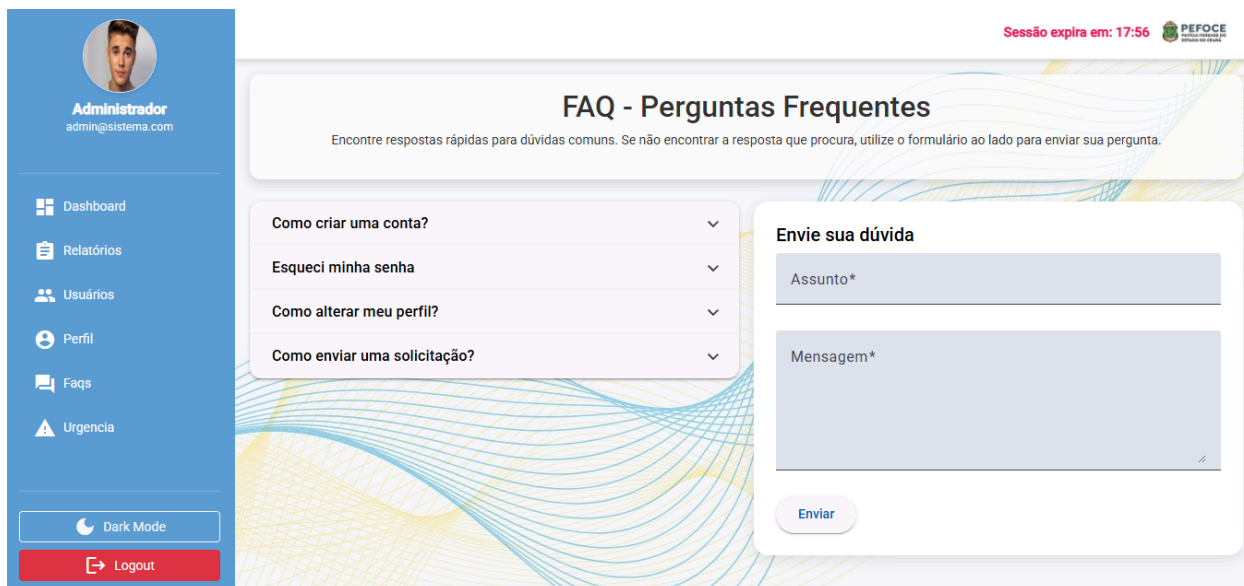
- Spinner (`mat-progress-spinner`) exibido enquanto os FAQs são carregados (simulado com timeout de 800ms).
- **Responsividade:**
 - Layout dividido em duas colunas em telas maiores (FAQs e formulário).
 - Colunas empilhadas em dispositivos móveis.

Funcionalidades Atuais

- **Visualização de FAQs:** apresenta lista de perguntas e respostas com expansão interativa.
- **Envio de dúvidas:** permite que usuários preencham um formulário e recebam feedback visual imediato (toast).
- **Suporte a dark mode:** cores e backgrounds adaptam-se ao tema selecionado.
- **Simulação de backend:** FAQs são carregadas localmente com delay para simular requisição HTTP.

Observações

- Dados de FAQs e envio de dúvidas não possuem persistência real; tudo é **local e temporário**.
- Essa tela é um template, não há ligação com o backend
- Mantém consistência visual com outras telas do dashboard, incluindo sombras, bordas arredondadas e tipografia.



Tela de Urgência

A **tela de Urgência** é destinada à gestão e organização de solicitações prioritárias pelos usuários com **permissão de Admin ou Perito**. Atualmente, **não há integração real com backend** para modificar dados; o objetivo é **apresentar informações simuladas** e servir como ideia para futuras funcionalidades de gestão.

Disponibilidade

- Permitida somente para usuários com perfil **Admin** ou **Perito**.
 - Não possui rota real no backend para atualização ou envio de dados.
 - Funciona como **painel de visualização e organização**.
-

Componentes Principais

1. Cards de Solicitações

- Cada solicitação é exibida em um card interativo:
 - Barra de status colorida (verde, amarelo, cinza) de acordo com o status.
 - Ícone circular numerado representando a posição na lista.
 - Conteúdo: título, protocolo, data, status e descrição (se houver).
- Cards são **arrastáveis** usando **CDK Drag and Drop** para reorganização.
- Suporte a **dark mode** com cores adaptativas para texto e background.

2. Seção de Gráficos

- **Gráfico de Status:** gráfico de pizza (**pie**) mostrando a quantidade de solicitações por status (ex.: Aberto, Em Andamento).
 - **Gráfico de Ocorrências por Mês:** gráfico de barras mostrando a quantidade de solicitações por mês.
 - Ambos gráficos usam **Chart.js** e se ajustam ao tema (dark/light mode)
-

Funcionalidades

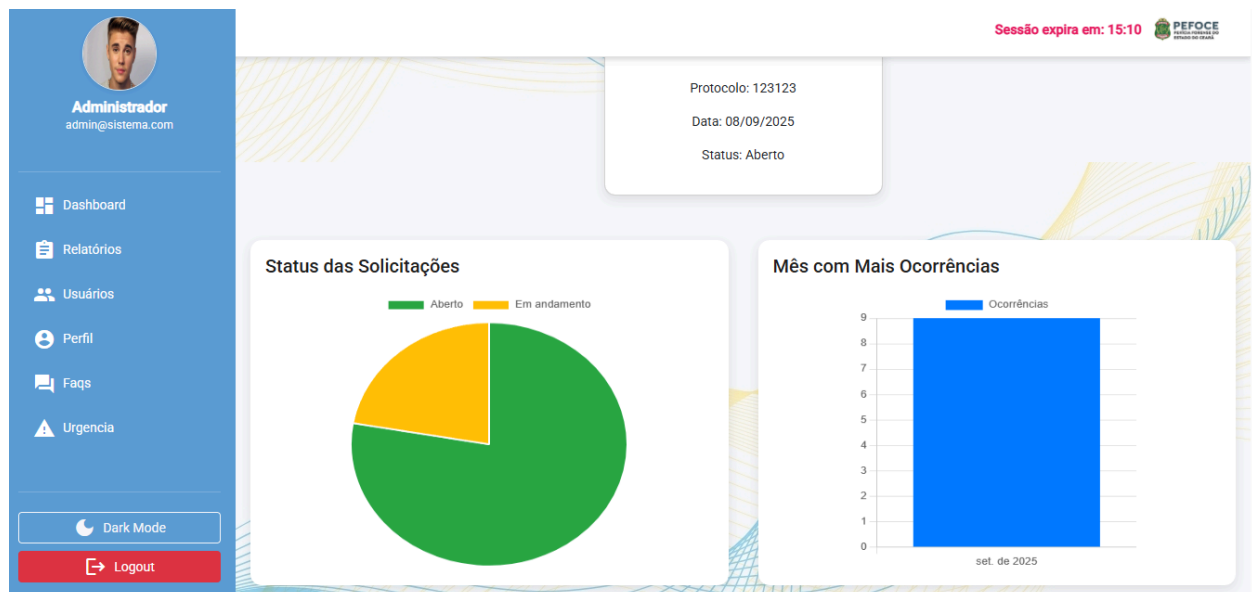
- **Carregamento de Solicitações:**
 - Simula requisição HTTP para recuperar dados.
 - Filtra apenas solicitações que não estão **fechadas**.
 - Formata datas para `DD/MM/YYYY` e mantém padrão ISO para outros usos.
 - **Reorganização de Cards:**
 - Cards podem ser arrastados e reposicionados na tela (`CdkDragDrop`).
 - A alteração de ordem é visual; ainda não salva no backend.
 - **Visualização de Detalhes:**
 - Clique em um card abre modal (`MatDialog`) com detalhes da solicitação.
 - **Status Colorido:**
 - `Aberto` : amarelo (`#ffc107`)
 - `Em andamento` : verde (`#28a745`)
 - Outros/Fechados: cinza (`#6c757d`)
 - **Gráficos Dinâmicos:**
 - Status das solicitações (pie chart).
 - Quantidade de ocorrências por mês (bar chart).
 - Responsivos e adaptáveis ao tema.
 - **Suporte a Dark Mode:**
 - Cores de cards, textos, barras e gráficos adaptam-se automaticamente.
-

Observações e Futuras Melhorias

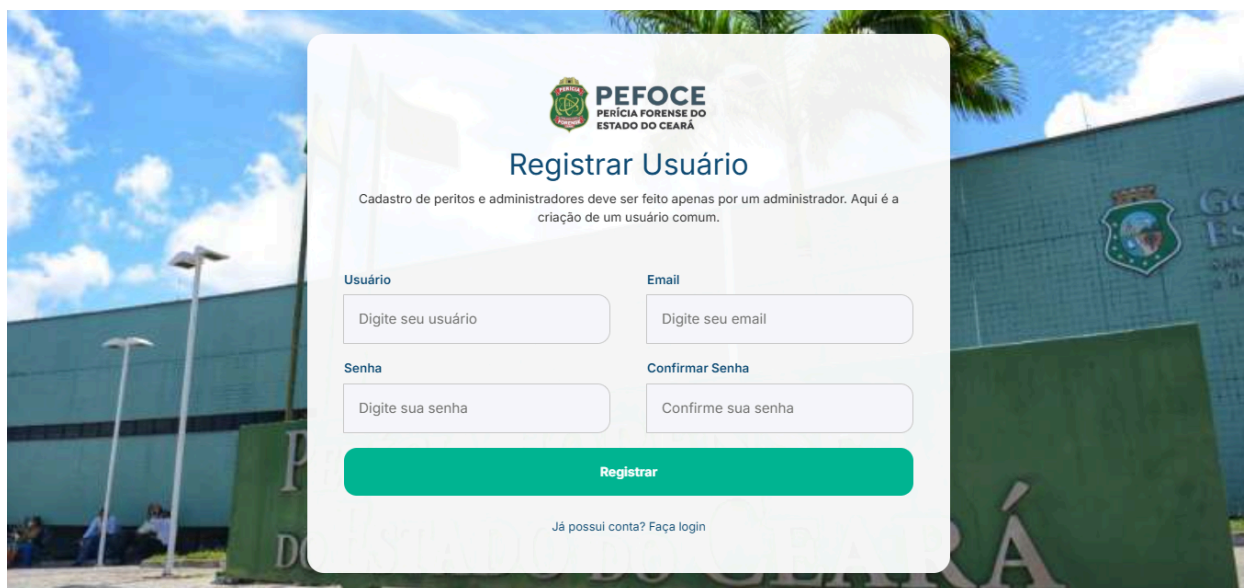
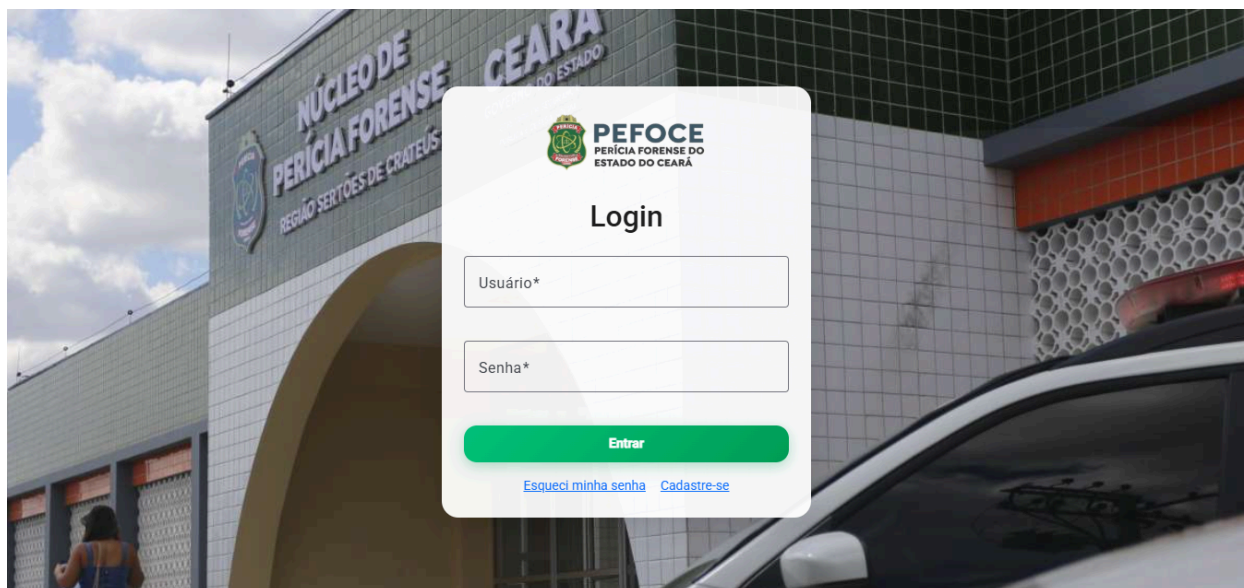
- Tela **conceitual** para organizar e priorizar solicitações de urgência.
- Possível integração futura com backend para:
 - Salvar ordem dos cards após drag and drop.
 - Permitir criação, edição e fechamento de solicitações.
 - Atualização em tempo real via WebSocket ou polling.

- Estrutura pronta para integração com dashboards de relatórios ou gestão operacional.





Telas de Autenticação





Considerações Finais

Gostaria de expressar meu entusiasmo em fazer parte da equipe da **Pefoce**. Estou muito motivada com esta oportunidade, não apenas para contribuir com minhas habilidades técnicas, mas também para **crescer profissionalmente e aprender com os desafios e experiências da instituição**.

Acredito que a Pefoce oferece um ambiente de constante aprendizado e evolução, e estou totalmente comprometida em agregar valor à equipe, absorver conhecimento, aprimorar minhas competências e **colaborar ativamente para o sucesso dos projetos e da organização**.

Estou confiante de que esta oportunidade será transformadora, permitindo-me desenvolver habilidades, enfrentar desafios reais e evoluir continuamente junto à Pefoce.

Atenciosamente,

Giulia Trevisan

